

Examen final de Logiciels de Calcul Scientifique

Aucun document n'est autorisé

Une grande importance sera attachée à la rigueur du raisonnement
 et à la clarté et au soin de la présentation.

Exercice 1. Les polynômes de Jacobi $P_n^{(\alpha, \beta)}(x)$ sont définis par :

$$P_n^{(\alpha, \beta)}(x) = \frac{1}{2^n} \sum_{k=0}^n \binom{n+\alpha}{k} \binom{n+\beta}{n-k} (x-1)^k (x+1)^{n-k}, \quad (1)$$

où α et β sont des paramètres réels positifs.

Ces polynômes sont associés à la fonction poids $w(x) = (1-x)^\alpha (1+x)^\beta$, $\alpha, \beta > -1$.

Les premiers termes sont donnés pour $\alpha = \beta = 0$ (polynômes de Legendre) :

$$P_0(x) = 1, \quad P_1(x) = x, \quad P_2(x) = \frac{1}{2}(3x^2 - 1), \quad P_3(x) = \frac{1}{2}(5x^3 - 3x).$$

1. Écrire une fonction Octave nommée `c=NbrComb(n,p)` qui prend en argument deux entiers n et p avec $p \leq n$ et qui retourne le nombre de combinaisons $\binom{n}{p} = \frac{n!}{k!(n-k)!}$. vous pouvez utiliser la fonction prédéfinie `factorial` de Octave.
2. Écrire un script Octave nommé `TracerJacobi` qui permet de tracer les polynômes de Jacobi $P_0^{(0,0)}$, $P_1^{(0,0)}$, $P_2^{(0,0)}$ et $P_3^{(0,0)}$ pour $\alpha = \beta = 0$.
3. Écrire une fonction Octave nommée `P=PolyJacobi(n, alpha, beta)` qui prend en argument un entier n et deux valeurs α et β et qui retourne le vecteur de coefficients de $P_n^{(\alpha, \beta)}$ en utilisant la formule (1). Pourriez vous proposer une autre fonction qui retourne l'expression symbolique de $P_n^{(\alpha, \beta)}$.
4. Pour une fonction f définie sur $[-1, 1]$, l'approximation orthogonale de degré n par les polynômes de Jacobi est donnée par :

$$f_n(x) = \sum_{k=0}^n c_k P_k^{(\alpha, \beta)}(x), \quad \text{ou} \quad c_k = \frac{\int_{-1}^1 f(x) P_k^{(\alpha, \beta)}(x) w(x) dx}{\int_{-1}^1 \left(P_k^{(\alpha, \beta)}(x) \right)^2 w(x) dx}.$$

- (a) Écrire une fonction Octave `P=JacobiCoeffs(f, n, alpha, beta)` qui retourne les coefficients c_k sous forme d'un vecteur P dont les composantes sont $c_0, c_1, \dots, c_n$.
 - (b) Écrire une fonction `P=ApproximationJacobi(f, n, alpha, beta)` qui retourne l'approximation orthogonale de degré n .
5. Écrire une fonction `TracerApproxJacobi(f, n, alpha, beta)` qui affiche la courbe de la fonction f et celle de son approximation par projection orthogonale.

Exercice 2. 1. Soit un polynôme de degré n donné par :

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0,$$

où $a_n \neq 0$. La matrice compagnon associée à ce polynôme est définie par :

$$C = \begin{pmatrix} 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & 1 \\ -\frac{a_0}{a_n} & -\frac{a_1}{a_n} & \dots & \dots & -\frac{a_{n-1}}{a_n} \end{pmatrix}.$$

- Écrire une fonction Octave `C = Compagnon(a)` qui prend en entrée un vecteur `a` contenant les coefficients $(a_0, a_1, \dots, a_n)$ du polynôme et retourne la matrice compagnon correspondante.
- Écrire une fonction Octave `C = Compagnon(a)` qui permet de vérifier que les valeurs propres de cette matrice sont les racines du polynôme.
- Écrire une partie de code permettant de tester votre fonction avec le polynôme suivant :

$$P(x) = x^3 - 3x^2 + 3x - 1.$$

- Écrire une fonction Octave nommée `[A, B, C] = Declaration(n)` qui prend en argument un entier n et retourne les matrices suivantes :

(a) A une matrice de taille n dont tous les éléments valent n .

(b) B est une matrice diagonale dont la diagonale est $(n, n-1, n-2, \dots, 1, 2, 3, \dots, n)$

$$(c) C = \begin{pmatrix} 1 & 2 & \dots & n & 1 \\ n+1 & n+2 & \dots & 2n & 2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ n(n-1)+1 & n(n-1)+2 & \dots & n^2 & n-1 \\ 1 & 2 & \dots & n & n \end{pmatrix}$$

- Écrire une fonction Octave nommée `[A, B, C] = mat1(a, b, c, P)` qui prend en argument a, b, c et un vecteur $P = (p_1, \dots, p_n)$ et qui retourne les matrices suivantes :

$$A = \begin{pmatrix} b & c & \dots & c \\ a & b & \ddots & \vdots \\ \vdots & \ddots & \ddots & c \\ a & \dots & a & b \end{pmatrix}, \quad B = \begin{pmatrix} p_1 & p_2 & p_3 & \dots & p_n \\ c & p_1 & p_2 & \dots & p_{n-1} \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & p_1 & p_2 \\ c & \dots & \dots & c & p_1 \end{pmatrix}, \quad C = \begin{pmatrix} p_n & p_{n-1} & \dots & \dots & p_1 \\ b & p_n & \ddots & & \vdots \\ & \ddots & \ddots & \ddots & \vdots \\ & & \dots & b & p_n & p_{n-1} \\ b & & & \dots & b & p_n \end{pmatrix}$$

(les tailles des matrices sont exactement le nombre de composante du vecteur P)

Bonne chance.